ACD Lab Programs

OBJECTIVE: Write a C program to identify whether a given line is a comment or not.

//a1.c

```
#include<stdio.h>
#include<conio.h>
void main()
{
char com[30];
int i=2,a=0;
clrscr();
printf("\n Enter comment:");
gets(com);

if(com[0]=='/')
{
        if(com[1]=='/')
        printf("\n It is a comment");
                else if(com[1]=='*')
                {
                        for(i=2;i<=30;i++)
                        {
                                if(com[i]=='*'&&com[i+1]=='/')
                                {
                                printf("\n It is a comment");
                                a=1;
                                break;
                                }
                        else
                        continue;
                        }
                if(a==0)
                printf("\n It is not a comment");
                }
        else
        printf("\n It is not a comment");
        }
else
printf("\n It is not a comment");
getch();
}
```

INPUT & OUTPUT:
Input: Enter comment: //hello
Output: It is a comment
Input: Enter comment: hello
Output: It is not a comment

OBJECTIVE: Write a C program to test whether a given identifier is valid or not

```
//a2.c
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
void main()
{
char a[10];
int flag, i=1;

printf("\n Enter an identifier:");
gets(a);
flag=1;
if(isalpha(a[0]))
{
        while(a[i]!='\0')
        {
                if(!isdigit(a[i])&&!isalpha(a[i]))
                { // Nut a Digit not a Alphbet
                flag=0;
                break;
                }
        i++;
        }
}

else
        {
        flag=0;
        }
if(flag==0)
{
printf("\n Not a valid identifier");
getchar();
}
else
{
printf("\n Valid identifier");
}
}
```

INPUT & OUTPUT:
Input: Enter an identifier: first
Output:
Valid identifier
Enter an identifier:1aqw
Not a valid identifier

OBJECTIVE: Write a C program to check for keyword.

```
//a3.c
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
 int flag=0,i,j;
 char
keyword[32][10]={"auto","double","int","struct","break","else","long
","switch", "case","enum","register","typedef","char","extern","return
","union","const","float","short","unsigned","continue","for","signed",
"void","default","goto","sizeof","voltile","do","if","static","while"},str
ing[10];

 clrscr();
 printf("Enter the String:\n");
 gets(string);

        for(i=0;i<32;i++)
         {
         j=strcmp(string, keyword[i]);
        if(j==0)
        flag=1;
         }

 if(flag==0)
       printf("%s is not a keyword.",string);
 else
        printf("%s is a keyword.",string);
 getch();
}
```

/* To Write a C program to develop a lexical analyzer to recognize a few patterns in C.*/
// file name: a4main.c //a4inter.c is created automatically that contains output

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
void main()
{
FILE *fi,*fo,*fop,*fk;
int flag=0, i=1;
char c, t, a[15], ch[15], file[20];
clrscr();
printf("\n Enter the File Name:");
scanf("%s",&file); // a4Input.C
fi=fopen(file,"r");
fo=fopen("a4inter.c","w");
fop=fopen("a4Oper.c","r");
fk=fopen("a4key.c","r");
c=getc(fi);
while(!feof(fi))
{
        if(isalpha(c)||isdigit(c)||(c=='['||c==']'||c=='.'==1))
        fputc(c,fo);
        else
        {
                if(c=='\n')
                fprintf(fo,"\t$\t");
                else
                fprintf(fo,"\t%c\t",c);
        }
        c=getc(fi);
}
fclose(fi);
fclose(fo);
fi=fopen("inter.c","r");
printf("\n Lexical Analysis");
fscanf(fi,"%s",a);
printf("\n Line: %d\n",i++);
while(!feof(fi))    // Main While
{
        if(strcmp(a,"$")==0)
        {
```

```
        printf("\n Line: %d \n",i++);
        fscanf(fi,"%s",a);
        }
        fscanf(fop,"%s",ch);
        while(!feof(fop))  // Nested While1
        {
                if(strcmp(ch,a)==0)
                {
                fscanf(fop,"%s",ch);
                printf("\t\t%s\t:\t%s\n",a,ch);
                flag=1;
                }
         fscanf(fop,"%s",ch);
        }                       // Nested While1End
        rewind(fop);
        fscanf(fk,"%s",ch);
        while(!feof(fk))   // Nested While2
        {
                if(strcmp(ch,a)==0)
                {
                fscanf(fk,"%k",ch);
                printf("\t\t%s\t:\tKeyword\n",a);
                flag=1;
                }
        fscanf(fk,"%s",ch);
        }                       // Nested While2 End
        rewind(fk);
        if(flag==0)
        {
                if(isdigit(a[0]))
                printf("\t\t%s\t:\tConstant\n",a);
                else
                printf("\t\t%s\t:\tIdentifier\n",a);
        }
        flag=0;
        fscanf(fi,"%s",a);
}       // Main While End
getch();
}
```

// file name: a4Key.C

```
int
void
main
char
if
for
while
else
printf
scanf
FILE
Include
stdio.h
conio.h
iostream.h
```

// file name: a4Oper.C

```
( open para
) closepara
{ openbrace
} closebrace
< lesser
> greater
" doublequote ' singlequote
: colon
; semicolon
# preprocessor
= equal
== asign
% percentage
^ bitwise
& reference
* star
+ add
- sub
\ backslash
/ slash
```

// file name: a4Input.C

```
#include "stdio.h"
#include "conio.h"
void main()
{
int a=10,b,c;
a=b*c;
getch();
}
```

======output of a4main.c

```
Enter the File Name:a4Input.c
Lexical Analysis
Line: 1
                #       :       preprocessor
                include :       Identifier
                "       :       doublequote
                stdio.h :       Keyword
                "       :       doublequote
Line: 2
                #       :       preprocessor
                include :       Identifier
                "       :       doublequote
                conio.h :       Keyword
                "       :       doublequote
Line: 3
                void    :       Keyword
                main    :       Keyword
                (       :       open
                )       :       closepara
Line: 4
                {       :       openbrace
Line: 5
                int     :       Keyword
                a       :       Identifier
                =       :       equal
                10      :       Constant
                ,       :       Identifier
                b       :       Identifier
                ,       :       Identifier
                c       :       Identifier
                ;       :       semicolon
Line: 6
                a       :       Identifier
                =       :       equal
                b       :       Identifier
                *       :       star
                c       :       Identifier
                ;       :       semicolon
Line: 7
                getch   :       Identifier
                (       :       open
                )       :       closepara
                ;       :       semicolon
Line: 8
                }       :       closebrace
Line: 9
                $       :       Identifier
```



==========0

Recursive Descent Parser

The given CFG grammar:

E -> E + E
E -> E * E
E -> (E) |id

Target string:

(id+id)*id

Un-ambiguous grammar:

E ->E + T|T
T -> T * F|F
F -> (E) |id

After removing left recursion from above grammar

E-> TE'
E'-> +TE' | null
T-> FT'
T'-> *FT'| null
F-> (E) | id

void E()	void T()	void Eprime()	void Tprime()

{
T();
Eprime();
}
	{
	check();
	Tprime();
	}
		{
		if(expression[count] == '+')
		{
		count++;
		T();
		Eprime();
		}
		}
			{
			if(expression[count] == '*')
			{
			count++;
			check();
			Tprime();
			}
			}

/* To Write a C program to Implement a top-down parser (Recursive Descent Parser) */

// file name: a5.c

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>

void Tprime();
void Eprime();
void E();
void check();
void T();

char expression[10];
int count, flag;

int main()
{
    count = 0;
    flag = 0;
    printf("\nEnter an Algebraic Expression:\t");
    scanf("%s", expression);
    E();
    if((strlen(expression) == count) && (flag == 0))
    {
        printf("\nThe Expression %s is Valid\n", expression);
    }
    else
    {
        printf("\nThe Expression %s is Invalid\n", expression);
    }
```

```
}

void E()
{
    T();
    Eprime();
}

void T()
{
    check();
    Tprime();
}


void Tprime()
{
    if(expression[count] == '*')
    {
        count++;
        check();
        Tprime();
    }
}
void check()
{
    if(isalnum(expression[count]))
    {
        count++;
    }
    else if(expression[count] == '(')
    {
        count++;
        E();
        if(expression[count] == ')')
        {
            count++;
        }
        else
        {
            flag = 1;
        }
    }
    else
    {
        flag = 1;
```

```
    }
}
void Eprime()
{
    if(expression[count] == '+')
    {
        count++;
        T();
        Eprime();
    }
}
```

Output:

```
Enter an Algebraic Expression:  (a+b)*c
The Expression (a+b)*c is Valid
```

==========0

Problem Statement : Design a Predictive Parser for the following grammar

G: {

E-> TE' ,
E' -> +TE' | 0,
T-> FT' ,
T'-> *FT'|0 ,
F-> (E) | id

}

AIM: To write a 'C' Program to implement for the Predictive Parser (Non Recursive Descent parser) for the given grammar.

Given the parse Table:

	id	+	*	(	)	$
E	E-> TE′			E-> TE′		
E′		E′ -> +TE′			E′->0	E′->0
T	T-> FT′			T-> FT′		
T′		T′->0	T′-> *FT′		T′->0	T′->0
F	F-> id			F->(E)		

ALGORITHM / PROCEDURE :

Input : string w$, Predictive Parsing table M

Output : A Left Most Derivation of the input string if it is valid , error otherwise.

Step1: Start

Step2: Declare a character array w[10] and Z as an array

Step3: Enter the string with $ at the end

Step4: if (A(w[z]) then increment z and check for (B(w[z])) and if satisfies increment z and check for 'd' if d is present then increment and check for (D(w*z+))

Step5: if step 4 is satisfied then the string is accepted

Else string is not

Step 6: Exit

// file name: a6.c

```
// ***IMPLEMENTATION OF PREDICTIVE / NON-RECURSIVE DESCENT PARSING *****//
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
char ch;
#define id 0
#define CONST 1
#define mulop 2
#define addop 3
#define op 4
#define cp 5
#define err 6
#define col 7
#define size 50
int token;
char lexbuff[size];
int lookahead=0;

int main()
{
        printf(" Enter the string :");
        gets(lexbuff);
        parser();
```

```
        return 0;
}

parser()
{
        if(E())
        printf("valid string");
        else
        printf("invalid string");
        getch();
        return 0;
}



E()
{
if(T())
        {
        if(EPRIME())
        return 1;
        else
        return 0;
        }
else
        return 0;
}

T()
{
        if(F())
        {
        if(TPRIME())
        return 1;
        else
        return 0;
        }
else
        return 0;
}

EPRIME()
{
```

```
token=lexer();
if(token==addop)
{
        lookahead++;
        if(T())
        {
        if(EPRIME())
                return 1;
        else
                return 0;
        }
        else
        return 0;
        }
else
        return 1;
}

TPRIME()
{
token=lexer();
if(token==mulop)
{
        lookahead++;
        if(F())
        {
                if(TPRIME())
                return 1;
                else
                return 0;
        }
        else return 0;
}
else return 1;
}

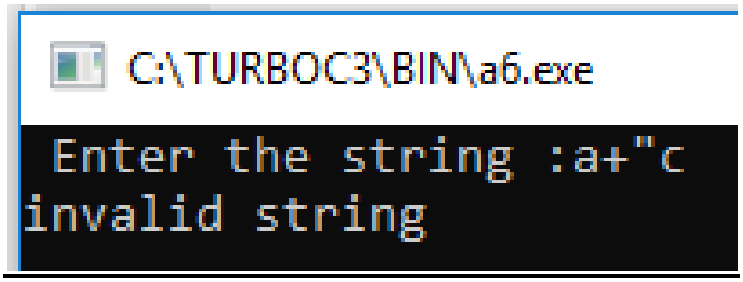
F()
{
token=lexer();
if(token==id)
return 1;
else
```

```
{
	if(token==4)
	{
		if(E())
		{
			if(token==5)
			return 1;
			else return 0;
		}
		else return 0;
		}
	else return 0;
	}
}

lexer()
{
if(lexbuff[lookahead]!='\n')
{
	while(lexbuff[lookahead]=='\t')
	lookahead++;
	if(isalpha(lexbuff[lookahead]))
	{
		while(isalnum(lexbuff[lookahead]))
		lookahead++;
		return(id);
	}
	else
	{
		if(isdigit(lexbuff[lookahead]))
		{
			while(isdigit(lexbuff[lookahead]))
			lookahead++;
			return CONST;
		}
		else
		{
```

```
                if(lexbuff[lookahead]=='+')
                {
                        return(addop);
                }
                else
                {
                        if(lexbuff[lookahead]=='*')
                        {
                        return(mulop);
                        }
                        else
                        {
                                if(lexbuff[lookahead]=='(')
                                {
                                        lookahead++;
                                        return(op);
                                }
                                else
                                {
                                        if(lexbuff[lookahead]==')')
                                        {
                                                return(op);
                                        }
                                        else
                                        {
                                                return(err);
                                        }
                                }
                        }
                }
        }
    }
}
else
return (col);
```

```
}
```



==========0

AIM : To write a C program to generate a three address code for a given expression.

// file name: a7.c

ALGORITHM:

Step1: Begin the program

Step2 : The expression is read from the file using a file pointer

Step3 : Each string is read and the total no. of strings in the file is calculated.

Step4: Each string is compared with an operator; if any operator is seen then the previous string and next string are concatenated and stored in a first temporary value and the three address code expression is printed

Step5 : Suppose if another operand is seen then the first temporary value is concatenated to the next string using the operator and the expression is printed.

Step6 : The final temporary value is replaced to the left operand value.

Step7 : End the program.

```
//PROGRAM: program for three address code generation
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>
struct three
{
char data[10],temp[7];
}s[30];

void main()
{
char d1[7],d2[7]="t";
int i=0,j=1,len=0;
FILE *f1,*f2;

f1=fopen("a6sum.txt","r");
f2=fopen("a6out.txt","w");
while(fscanf(f1,"%s",s[len].data)!=EOF)
len++; // Each string is read and the total no. of strings in the file is calculated.
itoa(j,d1,7);
strcat(d2,d1);
strcpy(s[j].temp,d2);
strcpy(d1,"");
```

```
strcpy(d2,"t");
// string are concatenated and stored in a first temporary value
if(!strcmp(s[3].data,"+"))
        {
        fprintf(f2,"%s=%s+%s",s[j].temp,s[i+2].data,s[i+4].data);
        j++;
        }
else if(!strcmp(s[3].data,"-"))
        {
        fprintf(f2,"%s=%s-%s",s[j].temp,s[i+2].data,s[i+4].data);
        j++;
        }
        // if another operand is seen then the first temporary value is concatenated
for(i=4;i<len-2;i+=2)
        {
        itoa(j,d1,7);
        strcat(d2,d1);
        strcpy(s[j].temp,d2);

        if(!strcmp(s[i+1].data,"+"))
                fprintf(f2,"\n%s=%s+%s",s[j].temp,s[j-1].temp,s[i+2].data);
        else if(!strcmp(s[i+1].data,"-"))
                fprintf(f2,"\n%s=%s-%s",s[j].temp,s[j-1].temp,s[i+2].data);
        strcpy(d1,"");
        strcpy(d2,"t");
        j++;
        }
fprintf(f2,"\n%s=%s",s[0].data,s[j-1].temp);
fclose(f1);
fclose(f2);
getch();
}
```

Input: a6sum.txt

```
out = in1 + in2 + in3 - in4
```

Output : a6out.txt

```
t1=in1+in2
t2=t1+in3
t3=t2-in4
out=t3
```

AIM : To write a C program to generate an assembly language instruction from three address code.

ALGORITHM:

Step1 : Begin the program

Step2 : The file containing the three address code is given as input to this program

Step3 : Each string is read from the file

Step4 : In an expression the string the left of the operator is printed with the 'LDA' string.

Step5 : Next if the operator is '+' then 'ADO' is printed else 'SUB' is printed followed by the string to the right of the operator.

Step6 : Then 'STA' string is printed followed by the string value left of the '=' operator

Step7 : .End of the program

// file name: a8.c

```
//PROGRAM: GENERATION OF ASSEMBLY LANGUAGE
INSTRUCTION FROM THREE ADDRESS CODE
#include<stdio.h>
#include<conio.h>
#include<string.h>
struct three
{
char data[10],temp[7];
}s[30];

void main()
{
char *d1,*d2;
int i=0,len=0;
FILE *f1,*f2;

f1=fopen("exe1.txt","r");
f2=fopen("exe2.txt","w");

while(fscanf(f1,"%s",s[len].data)!=EOF)
```

```
len++;

for(i=0;i<=len;i++)
        {
          if(!strcmp(s[i].data,"="))
          {
          fprintf(f2,"\nLDA\t%s",s[i+1].data);
                    if(!strcmp(s[i+2].data,"+"))
                          fprintf(f2,"\nADD\t%s",s[i+3].data);

                    if(!strcmp(s[i+2].data,"-"))
                    fprintf(f2,"\nSUB\t%s",s[i+3].data);

          fprintf(f2,"\nSTA\t%s",s[i-1].data);
          }
    }
fclose(f1);
fclose(f2);
getch();
 }
```

Input: exe1.txt

```
t1 = in1 + in2
t2 = t1 + in3
t3 = t2 - in4
out = t3
```

Output: exe2.txt

```
LDA    in1
ADD   in2
STA    t1
LDA    t1
ADD   in3
STA    t2
LDA    t2
SUB    in4
STA    t3
LDA    t3
STA    out
```

==========0

==========0

6. Generate an Assembly code for the given arithmetic expression.

==========0

7. Implement the code optimization algorithms

==========0